Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение высшего образования «Казанский (Приволжский) федеральный университет»
Институт вычислительной математики и информационных технологий
Кафедра прикладной математики искусственного интеллекта

ОТЧЕТ ПО ПРОЕКТУ

по дисциплине «Анализ данных на Python»

МНОГОСЛОЙНЫЙ ПЕРСЕПТРОН

Выполнил:
Скворцов В.В. студент
группы 09-322

2026

Содержание

1. Постановка задачи и требования 3
2. Исходные данные и предварительный анализ 3
3. Подготовка данных и разделение выборки 4
3.1. Стратифицированное разделение 4
3.2. Стандартизация признаков 5
4. Архитектура многослойного персептрона 5
4.1. Инициализация весов 6
5. Математическая модель обучения 7
5.1. Прямое распространение 7
5.2. Softmax на выходном слое 7
5.3. Функция потерь 8
5.4. Выкладка градиента для softmax + cross-entropy 8
5.5. Обратное распространение 9
5.6. Градиентный спуск 9
6. Реализация трех программ 10
6.1. split_data.py 10
6.2. train_mlp.py 10
6.3. predict.py 11
7. Настройки эксперимента и численные выкладки 12
8. Результаты обучения и прогнозирования 13
8.1. Кривые обучения 13
8.2. Итоговое прогнозирование 13
8.3. Формат predictions.csv 14
9. Заключение 14

1. Постановка задачи и требования

Цель проекта — реализовать многослойный персептрон без использования библиотек, предоставляющих готовые алгоритмы обучения.

Необходимо решить задачу бинарной классификации: по 30 числовым характеристикам объекта предсказать диагноз B (benign, доброкачественный) или M (malignant, злокачественный). Нейронная сеть должна иметь не менее двух скрытых слоев, а на выходном слое должна быть реализована функция softmax, формирующая вероятностное распределение по двум классам.

Проект разделен на три независимые программы:

- split_data.py — анализ исходного датасета и воспроизводимое разделение на обучающую и проверочную выборки;
- train_mlp.py — обучение многослойного персептрона методом обратного распространения ошибки и градиентного спуска с сохранением модели;
- predict.py — загрузка сохраненной модели, прогнозирование и оценка результата бинарной кросс-энтропией.

Дополнительно на каждой эпохе выводятся значения функции потерь и точности для train и validation, а после обучения строятся две кривые: loss и accuracy.

2. Исходные данные и предварительный анализ

Исходный набор содержит 569 объектов и 32 столбца. Первый столбец содержит идентификатор объекта, второй — метку класса, оставшиеся 30 столбцов являются числовыми признаками. Все 569 значений первого столбца уникальны, поэтому идентификатор не используется как входной признак сети.

Распределение классов: B — 357 объектов (62.74%), M — 212 объектов (37.26%). Это умеренный дисбаланс, который был учтен при разбиении: выборки формировались стратифицированно, отдельно внутри каждого класса.

Масштабы признаков существенно различаются: стандартные отклонения лежат примерно в диапазоне от 0.00264374 до 568.856, а общий диапазон значений — от 0 до 4254. При прямой подаче таких величин признаки с большим численным масштабом могли бы непропорционально влиять на градиенты. Поэтому перед обучением выполняется стандартизация.

Признак	Mean	Std	Min	Max

Признак	Mean	Std	Min	Max
feature_01	14.1273	3.52095	6.981	28.11
feature_02	19.2896	4.29725	9.71	39.28
feature_03	91.969	24.2776	43.79	188.5
feature_04	654.889	351.605	143.5	2501
feature_05	0.0963603	0.0140518	0.05263	0.1634

Таблица 1 — Статистика первых пяти числовых признаков

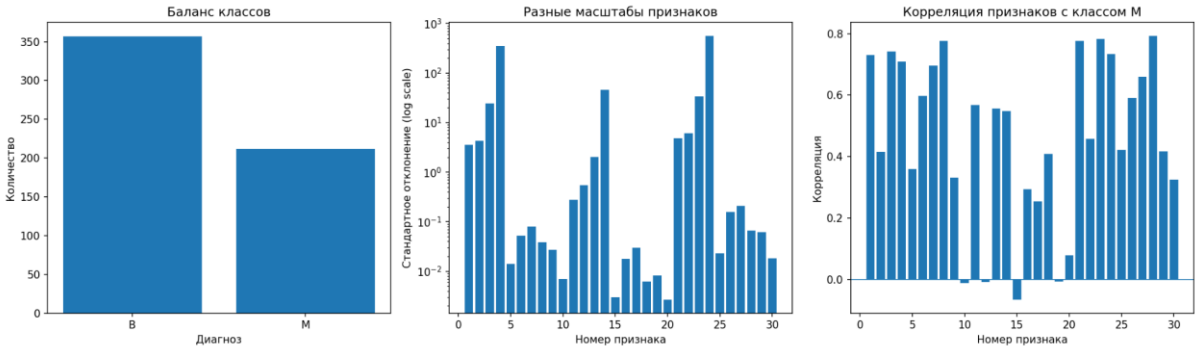


Рисунок 1 — Предварительный визуальный анализ набора данных

На рисунке 1 слева показан баланс классов, в центре — стандартные отклонения признаков в логарифмическом масштабе, справа — корреляция каждого признака с бинарной меткой M. Видно, что часть признаков существенно связана с диагнозом, а масштабы признаков различаются на несколько порядков. Это подтверждает необходимость стандартизации.

3. Подготовка данных и разделение выборки

3.1. Стратифицированное разделение

Разделение выполняется программой split_data.py. Для каждого класса индексы объектов перемешиваются отдельно, после чего 80% объектов каждого класса попадают в обучающую часть, а остальные — в проверочную. Затем обе части дополнительно перемешиваются. Везде используется seed=42, что обеспечивает повторяемость результата.

Фрагмент кода — Стратифицированное разделение

```
for label in ("B", "M"):
    idx = np.flatnonzero(y == label)
    rng.shuffle(idx)
    n_train = int(round(len(idx) * train_ratio))
    train_indices.extend(idx[:n_train].tolist())
    valid_indices.extend(idx[n_train:].tolist())

rng.shuffle(train_indices)
rng.shuffle(valid_indices)
```

В результате получено 456 объектов в обучающей выборке и 113 объектов в проверочной. В train: B=286, M=170; в validation: B=71, M=42. Доли классов остаются практически такими же, как в исходном наборе.

3.2. Стандартизация признаков

Среднее значение и стандартное отклонение каждого признака вычисляются только на обучающей выборке. Эти же параметры затем применяются к validation и к данным при прогнозировании. Такое решение предотвращает утечку информации из проверочной выборки в процесс обучения.

$$x'_j = (x_j - \mu_j) / \sigma_j \qquad (1)$$

где μ_j и σ_j — среднее значение и стандартное отклонение j-го признака, вычисленные по train. Если стандартное отклонение численно близко к нулю, в коде оно заменяется на 1, чтобы избежать деления на ноль.

Фрагмент кода — Стандартизация

```
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)
std = np.where(std < 1e-12, 1.0, std)
X_train = (X_train - mean) / std
X_valid = (X_valid - mean) / std
```

4. Архитектура многослойного персептрона

По умолчанию используется архитектура 30 → 24 → 24 → 2. Входной слой соответствует 30 числовым признакам. Два скрытых слоя содержат по 24 нейрона. Выходной слой содержит два нейрона — по одному для классов B и M.

Слой	Размер входа	Число нейронов	Активация	Параметры
Скрытый 1	30	24	sigmoid	30×24 + 24 = 744
Скрытый 2	24	24	sigmoid	24×24 + 24 = 600
Выходной	24	2	softmax	24×2 + 2 = 50

Таблица 2 — Архитектура сети и число обучаемых параметров

Всего сеть содержит 744 + 600 + 50 = 1394 обучаемых параметра.

4.1. Инициализация весов

Для сигмоидальной активации используется Xavier/Glorot uniform. Он ограничивает дисперсию сигналов и градиентов при прохождении через слои и лучше подходит для sigmoid/tanh, чем произвольная инициализация с большим разбросом.

$$\text{limit} = \sqrt{(6 / (\text{fan_in} + \text{fan_out}))}, \qquad W \sim U(-\text{limit}, \text{limit}) \qquad (2)$$

Смещения инициализируются нулями.

Фрагмент кода — Инициализация весов и смещений

```
for fan_in, fan_out in zip(layer_sizes[:-1], layer_sizes[1:]):
    if activation == "relu" and fan_out != layer_sizes[-1]:
        W = self.rng.normal(
            0.0, np.sqrt(2.0 / fan_in), size=(fan_in, fan_out)
        )
    else:
        limit = np.sqrt(6.0 / (fan_in + fan_out))
        W = self.rng.uniform(-limit, limit, size=(fan_in, fan_out))

    b = np.zeros((1, fan_out), dtype=np.float64)
```

5. Математическая модель обучения

5.1. Прямое распространение

Для каждого полносвязного слоя сначала вычисляется линейная комбинация входов, затем к ней применяется функция активации. Для слоя l:

$$z^{(l)} = a^{(l-1)} W^{(l)} + b^{(l)} \quad (3)$$

$$a^{(l)} = \varphi(z^{(l)}) \quad (4)$$

Фрагмент кода — Прямое распространение

```
def forward(self, X, keep_cache=False):
    activations = [X]
    pre_activations = []
    a = X

    for i in range(len(self.weights) - 1):
        z = a @ self.weights[i] + self.biases[i]
        a = activation_forward(z, self.activation)
        pre_activations.append(z)
        activations.append(a)

    logits = a @ self.weights[-1] + self.biases[-1]
    probabilities = softmax(logits)
    ...
    return probabilities
```

В скрытых слоях по умолчанию используется сигмоида:

$$\sigma(z) = 1 / (1 + e^{-z}) \quad (5)$$

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)) \quad (6)$$

Сигмоида реализована в численно устойчивой форме: для положительных и отрицательных z используются разные выражения, что снижает риск переполнения exp.

5.2. Softmax на выходном слое

Выходные логиты преобразуются в вероятности двух классов функцией softmax:

$$p_k = \exp(z_k) / \Sigma_j \exp(z_j) \quad (7)$$

Для численной устойчивости перед экспонентой из всех логитов строки вычитается максимальный логит. Такая операция не изменяет softmax, но предотвращает переполнение при больших значениях.

Фрагмент кода — Численно устойчивый softmax

```
shifted = logits - np.max(logits, axis=1, keepdims=True)
exp_values = np.exp(shifted)
probabilities = exp_values / np.sum(exp_values, axis=1, keepdims=True)
```

5.3. Функция потерь

В задаче два класса. Пусть y=1 для класса M и y=0 для класса B. Тогда бинарная кросс-энтропия по вероятности класса M имеет вид:

$$L = -(1/N) \Sigma_i [y_i \ln(p_i,M) + (1-y_i) \ln(1-p_i,M)] \quad (8)$$

Поскольку после softmax pB + pM = 1, выражение 1−pM равно pB. Поэтому для двух классов указанная BCE совпадает с обычной кросс-энтропией по двум softmax-выходам. Перед логарифмом вероятность ограничивается диапазоном $[10^{-12}, 1-10^{-12}]$, что исключает log(0).

Фрагмент кода — Бинарная кросс-энтропия

```
def binary_cross_entropy(y, probabilities):
    p_m = np.clip(probabilities[:, 1], 1e-12, 1.0 - 1e-12)
    y = y.astype(np.float64)
    return float(-np.mean(
        y * np.log(p_m) + (1.0 - y) * np.log(1.0 - p_m)
    ))
```

5.4. Выкладка градиента для softmax + cross-entropy

Для одного объекта с one-hot вектором y функция потерь записывается как $L = -\Sigma_k y_k \ln(p_k)$. Производная softmax по логиту известна в виде $\partial p_k/\partial z_j = p_k(\delta_{kj} - p_j)$. После применения правила цепочки члены сокращаются, и получается компактное выражение:

$$\partial L / \partial z_j = p_j - y_j \quad (9)$$

Для mini-batch размера N в реализации градиент усредняется:

$$\delta^{(out)} = (P - Y) / N \quad (10)$$

Это ключевое упрощение позволяет не выписывать полную матрицу Якоби softmax для каждого объекта.

5.5. Обратное распространение

Для выходного слоя сначала вычисляются градиенты весов и смещений:

$$\partial L/\partial W^{(l)} = (a^{(l-1)})^T \delta^{(l)} \quad (11)$$

$$\partial L/\partial b^{(l)} = \Sigma_i \delta_i^{(l)} \quad (12)$$

Затем ошибка распространяется в предыдущий скрытый слой:

$$\delta^{(l)} = [\delta^{(l+1)} (W^{(l+1)})^T] \odot \varphi'(z^{(l)}) \quad (13)$$

Операция $\odot$ обозначает поэлементное умножение. Таким образом, backpropagation является последовательным применением правила производной сложной функции от выхода к входу.

Фрагмент кода — Обратное распространение

```
def backward(self, X_batch, y_batch):
    probabilities, activations, pre_activations = \
        self.forward(X_batch, keep_cache=True)
    y_oh = one_hot(y_batch, 2)
    batch_size = len(X_batch)

    delta = (probabilities - y_oh) / batch_size
    grad_weights[-1] = activations[-2].T @ delta
    grad_biases[-1] = np.sum(delta, axis=0, keepdims=True)

    for layer in range(len(self.weights) - 2, -1, -1):
        delta = delta @ self.weights[layer + 1].T
        z = pre_activations[layer]
        a = activations[layer + 1]
        delta *= activation_derivative(z, a, self.activation)
        grad_weights[layer] = activations[layer].T @ delta
        grad_biases[layer] = np.sum(delta, axis=0, keepdims=True)

    return grad_weights, grad_biases
```

5.6. Градиентный спуск

После вычисления градиентов параметры каждого слоя обновляются по правилу mini-batch gradient descent:

$$W \leftarrow W - \eta \cdot \partial L/\partial W, \qquad b \leftarrow b - \eta \cdot \partial L/\partial b \tag{14}$$

Здесь η — learning rate. В эксперименте η=0.05. Перед каждой эпохой обучающие объекты воспроизводимо перемешиваются, после чего сеть обновляет параметры по небольшим пакетам.

Фрагмент кода — Шаг градиентного спуска

```
def step(self, grad_weights, grad_biases, learning_rate):
    for i in range(len(self.weights)):
        self.weights[i] -= learning_rate * grad_weights[i]
        self.biases[i] -= learning_rate * grad_biases[i]
```

6. Реализация трех программ

6.1. split_data.py

Программа проверяет число столбцов, допустимость диагнозов B/M, преобразование всех признаков к float и отсутствие NaN/inf. Затем печатает статистику, строит рисунок 1 и выполняет стратифицированное разбиение. Обычный запуск автоматически ищет data.csv.

6.2. train_mlp.py

Программа загружает train/validation, выполняет стандартизацию по параметрам train, создает MLP и обучает его в течение заданного числа эпох. На каждой эпохе после обновления весов вычисляются четыре метрики: train loss, train accuracy, validation loss и validation accuracy.

Фрагмент кода — Основной цикл обучения

```
for epoch in range(1, epochs + 1):
    indices = rng.permutation(n)
    for start in range(0, n, batch_size):
        batch_idx = indices[start:start + batch_size]
        grad_w, grad_b = model.backward(X_train[batch_idx], y_train[batch_idx])
        model.step(grad_w, grad_b, learning_rate)

    train_probs = model.forward(X_train)
    valid_probs = model.forward(X_valid)
```

По завершении обучения сохраняется NPZ-файл. В нем находятся размеры слоев, название активации, mean/std каждого признака, имена классов, все матрицы весов и векторы смещений. Это позволяет полностью восстановить модель без повторного обучения.

Фрагмент кода — Сохранение модели

```
def save_model(path, model, feature_mean, feature_std):
    payload = {
        "layer_sizes": np.asarray(model.layer_sizes, dtype=np.int64),
        "activation": np.asarray(model.activation),
        "feature_mean": feature_mean.astype(np.float64),
        "feature_std": feature_std.astype(np.float64),
        "class_names": CLASS_NAMES,
    }

    for i, (W, b) in enumerate(zip(model.weights, model.biases)):
        payload[f"W{i}"] = W
        payload[f"b{i}"] = b

    np.savez_compressed(path, **payload)
```

6.3. predict.py

Программа прогнозирования загружает saved_model.npz, восстанавливает архитектуру и параметры стандартизации, читает входной набор и выполняет только forward propagation. Для размеченного validation дополнительно вычисляются BCE, accuracy и матрица ошибок.

Фрагмент кода — Загрузка сохраненной модели

```
def load_model(path):
    with np.load(path, allow_pickle=False) as data:
        layer_sizes = data["layer_sizes"].astype(int).tolist()
        activation = str(data["activation"].item())
        feature_mean = data["feature_mean"].astype(np.float64)
        feature_std = data["feature_std"].astype(np.float64)
        class_names = data["class_names"].astype(str)

        n_layers = len(layer_sizes) - 1
        weights = [data[f"W{i}"] for i in range(n_layers)]
        biases = [data[f"b{i}"] for i in range(n_layers)]

    return layer_sizes, activation, feature_mean, feature_std, \
        class_names, weights, biases
```

Фрагмент кода — Прогнозирование сохраненной моделью

```
X = (X - feature_mean) / feature_std
probabilities = forward(X, activation, weights, biases)
predictions = np.argmax(probabilities, axis=1)

save_predictions(
    "predictions.csv", ids, probabilities, class_names, y
)
```

7. Настройки эксперимента и численные выкладки

Параметр	Значение
Train/validation	80% / 20% (стратифицированно)
Архитектура	30 → 24 → 24 → 2
Скрытая активация	sigmoid
Выходная активация	softmax
Learning rate	0.05
Batch size	16
Эпохи	120
Функция потерь	binary cross-entropy

Таблица 3 — Параметры обучения

Число mini-batch на эпоху равно ceil(456 / 16) = 29. За 120 эпох выполняется 29 × 120 = 3480 обновлений параметров. Этого оказалось достаточно.

Число обучаемых параметров равно 1394. Для данного набора это умеренный размер модели: он позволяет нелинейно разделить классы, но не создает чрезмерно большую сеть относительно 456 обучающих объектов.

8. Результаты обучения и прогнозирования

8.1. Кривые обучения

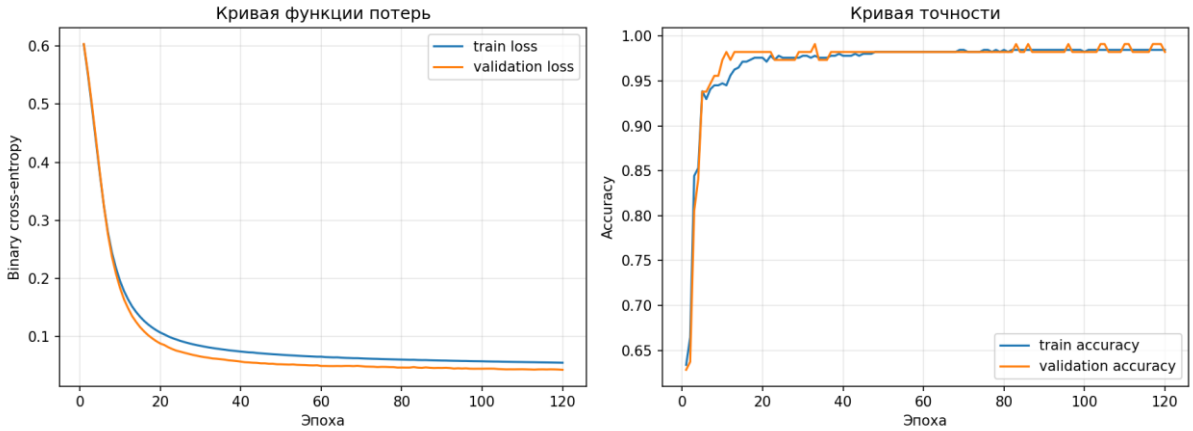


Рисунок 2 — Кривые функции потерь и точности

На первых эпохах loss быстро уменьшается, а accuracy возрастает. Затем изменения становятся небольшими. На 120-й эпохе train loss = 0.055287, train accuracy = 0.9846, validation loss = 0.043132, validation accuracy = 0.9823. Кривые train и validation близки, поэтому выраженных признаков переобучения по данному разбиению не наблюдается.

8.2. Итоговое прогнозирование

На проверочном наборе из 113 объектов модель правильно классифицировала 111 объектов и ошиблась на 2. Итоговая accuracy составляет 0.9823. Пересчитанное значение binary cross-entropy равно 0.043132.

	pred B	pred M
actual B	70	1
actual M	1	41

Таблица 4 — Матрица ошибок на validation

Получено две ошибки: один объект класса B был отнесен к M и один объект класса M был отнесен к B. Значение accuracy согласуется с матрицей ошибок: (70 + 41) / 113 = 111 / 113 ≈ 0.9823.

8.3. Формат predictions.csv

Для каждого объекта сохраняются идентификатор, предсказанный класс, вероятности B и M, а при наличии истинной метки — столбец actual. Сумма prob_B + prob_M равна 1 с точностью до округления, что является прямым следствием softmax.

id	predicted	prob_B	prob_M	actual
898690	B	0.997961	0.002039	B
8911834	B	0.993164	0.006836	B
873593	M	0.000318	0.999682	M
8911670	M	0.013058	0.986942	M

Таблица 5 — Пример строк файла predictions.csv

9. Заключение

В работе с нуля реализован многослойный персептрон для бинарной классификации. Были самостоятельно запрограммированы полносвязные слои, функции активации, softmax, бинарная кросс-энтропия, прямое и обратное распространение, вычисление градиентов и mini-batch gradient descent.

Перед обучением выполнен анализ данных и обоснована стандартизация. Разделение на train и validation сделано стратифицированно и воспроизводимо с seed=42. Параметры стандартизации вычисляются только по обучающим данным, что исключает утечку информации.

Архитектура 30 → 24 → 24 → 2 после 120 эпох достигла точности 98.23% на validation и binary cross-entropy 0.043132. Из 113 объектов ошибочно классифицировано только 2. Полученные кривые обучения демонстрируют устойчивую сходимость и отсутствие выраженного расхождения train/validation.

Проект состоит из трех программ и сохраняет модель в переносимом NPZ-файле, благодаря чему этап прогнозирования не зависит от повторного обучения. Реализация демонстрирует основные принципы работы многослойных нейронных сетей.

10. Дополнение Сравнение функций активации

В основном отчёте рассмотрен базовый запуск с sigmoid. В дополнении представлены результаты дополнительных запусков с ReLU и tanh и их сопоставление с базовым вариантом. Для обоих вариантов использована архитектура 30 → 24 → 24 → 2, выход softmax, 456 объектов train, 113 объектов validation, 120 эпох и seed = 42.

Результаты с ReLU

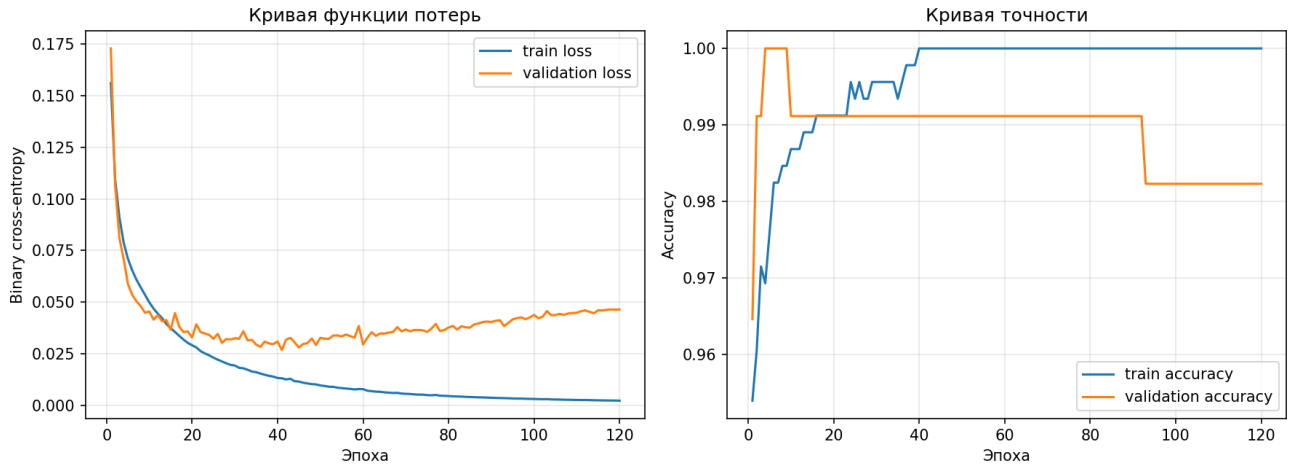


Рисунок 3 - Кривые функции потерь и точности при ReLU

При ReLU ошибка на train быстро уменьшается. Train accuracy достигает 100% на эпохе 40 и сохраняется до конца. На эпохе 120 train BCE = 0.002175, validation BCE = 0.046402, validation accuracy = 0.9823.

Минимальный validation BCE равен 0.026764 на эпохе 41; validation accuracy в этот момент составляет 0.9912. К концу обучения validation BCE возрастает на 0.019638, или на 73.37% относительно минимума, хотя train BCE продолжает снижаться. С эпохи 93 validation accuracy уменьшается с 0.9912 до 0.9823. Такая динамика указывает на отчётливое позднее переобучение.

В итоговой матрице ошибок строки actual B и actual M имеют значения (71, 0) и (2, 40), столбцы соответствуют predicted B и predicted M. Правильно классифицировано 111 из 113 объектов: (71 + 40) / 113 ≈ 0.9823. Обе ошибки - отнесение объектов M к классу B.

Результаты с tanh

При tanh обучение также быстро снижает loss, но validation-метрики заметнее колеблются. Train accuracy впервые достигает 100% на эпохе 59, а начиная с эпохи 79 сохраняется на этом уровне.

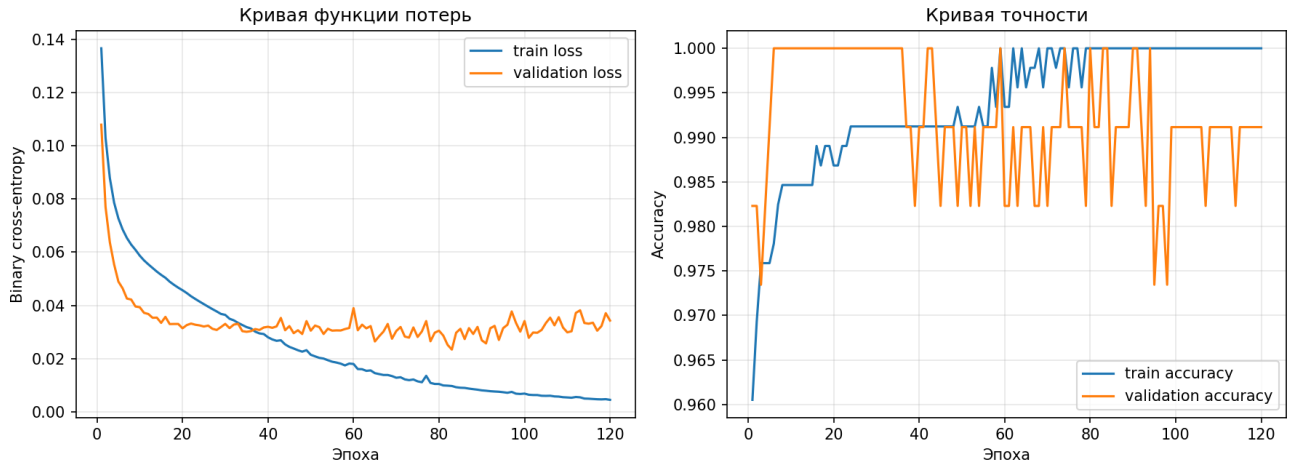


Рисунок 4 - Кривые функции потерь и точности при tanh

На эпохе 120 train BCE = 0.004537, train accuracy = 1.0000, validation BCE = 0.034298, validation accuracy = 0.9912. Матрица ошибок содержит строки (70, 1) для actual B и (0, 42) для actual M: правильно определено 112 объектов из 113, accuracy = (70 + 42) / 113 ≈ 0.9912. Единственная ошибка - B → M.

Глобальный минимум validation BCE равен 0.023402 на эпохе 83; validation accuracy в этой эпохе равна 1.0000. На последней эпохе BCE выше минимума на 0.010896, или на 46.56%. При этом train loss продолжает уменьшаться. Средний validation BCE на эпохах 101-120 равен 0.032824, тогда как на эпохах 81-100 - 0.030065. Это признаки умеренного позднего ухудшения обобщения; рост не является монотонным, а эпоху 83 нельзя считать точно установленной границей начала переобучения.

Колебания validation accuracy связаны с размером выборки: один объект из 113 меняет её на 0.885 процентного пункта. BCE учитывает вероятности и может меняться без изменения предсказанных классов; 100% accuracy не требует нулевой BCE.

Сопоставление результатов и общий вывод

В таблице 6 финальные метрики эпохи 120 отделены от минимального validation BCE и соответствующей эпохи: последняя модель может уступать лучшей по validation loss.

Показатель	sigmoid	ReLU	tanh
Train BCE на эпохе 120	0.055287	0.002175	0.004537
Train accuracy на эпохе 120	98.46%	100%	100%
Validation BCE на эпохе 120	0.043132	0.046402	0.034298
Validation accuracy на эпохе 120	98.23%	98.23%	99.12%
Минимальный validation BCE	0.043132	0.026764	0.023402
Эпоха минимума validation BCE	120	41	83

Таблица 6 - Финальные результаты и минимумы validation BCE

Sigmoid обучается медленнее: validation BCE впервые становится ниже 0.05 на эпохе 60; при ReLU - на эпохе 8, при tanh - на эпохе 5. Однако у sigmoid минимум достигнут на последней эпохе и устойчивого позднего ухудшения за 120 эпох не наблюдается.

В данном эксперименте tanh даёт лучшие финальные validation BCE и accuracy: 0.034298 и 99.12%. У ReLU наиболее отчётливо позднее переобучение. Sigmoid медленнее обучается, но плавнее изменяет validation loss. Результат зависит от настроек, включая соответствующую активации инициализацию; один seed и одно разбиение не доказывают универсального превосходства функции